

# QAgent: An LLM-based Multi-Agent System for Autonomous OpenQASM Programming

Zhenxiao Fu\*, Lei Jiang\*, Yilun Xu†, Gang Huang†, Fan Chen\*

\*Indiana University Bloomington

†Lawrence Berkeley National Laboratory

\*{zhfu, jiang60, fc7}@iu.edu

†{yilunxu, ghuang}@lbl.gov

**Abstract**—Programming quantum circuits at the OpenQASM level is essential for achieving hardware-aware optimization and reliable execution on noisy intermediate-scale quantum (NISQ) devices, yet it remains challenging due to the need for domain-specific planning, iterative code synthesis, and low-level calibration. In this paper, we present QAgent, the first autonomous multi-agent framework for end-to-end OpenQASM code generation. QAgent integrates schema-aware task planning, example- and tool-driven code synthesis, and hardware-aware calibration within a unified planning–synthesis–calibration workflow. The system leverages retrieval-augmented generation (RAG) to access structured kernel knowledge, examples, and backend constraints, and employs coordinated multi-agent reasoning with iterative execution feedback to ensure correctness. We evaluate QAgent on 12 representative quantum kernels and their compositions across five large language models (LLMs). Results show that QAgent improves Pass@1 accuracy by 47–70% on single-kernel tasks and achieves over 88% accuracy on multi-kernel workflows for large models, substantially outperforming existing baselines. Furthermore, under realistic hardware frequency drift, QAgent maintains near-unit execution fidelity through automated calibration, whereas SDK-based LLM methods suffer significant degradation. These results demonstrate that integrating planning, synthesis, and calibration is critical for reliable quantum program generation. The implementation of QAgent is open-sourced at <https://github.com/fuzhenxiao/QAgent>.

**Index Terms**—OpenQASM Programming, LLM Agents, RAG

## I. INTRODUCTION

Noisy Intermediate-Scale Quantum (NISQ) devices have demonstrated quantum advantage in classically intractable domains, including physical simulation, chemistry, and combinatorial optimization. These capabilities are enabled in large part by the Open Quantum Assembly Language (OpenQASM) [1], which provides a standardized interface between quantum software development kits (SDKs), such as Qiskit [2] and PennyLane [3], and heterogeneous quantum hardware platforms [4], [5]. While modern SDKs offer high-level abstractions for quantum circuit design and can export subsets of OpenQASM, the generated circuits often omit hardware-specific decompositions, pulse-level optimizations, and calibration data due to limited access to low-level device control. As a result, direct OpenQASM programming remains critical for fine-grained compiler optimization [6], experimental calibration and tuning [7], and hardware-aware pulse-level execution [8].

OpenQASM programming remains challenging, particularly for non-experts, due to its reliance on low-level hardware details such as qubit frequencies, temporal drift, control-port

mappings, and pulse scheduling [9], [10]. To address these complexities, OpenQASM development typically follows a structured, multi-stage workflow [1]. In the *planning* stage, high-level algorithmic objectives are decomposed into smaller computational tasks. Suitable circuit kernels [11] are then selected and mapped onto available hardware resources, taking into account device constraints and kernel-specific requirements. The subsequent *code synthesis* stage generates the corresponding OpenQASM implementation by defining quantum and classical registers, composing kernel operations, and assembling the full circuit. While canonical algorithms such as Deutsch–Jozsa can be synthesized using well-established templates, more complex or parameterized kernels often require specialized libraries and arithmetic abstractions. This stage typically concludes with systematic verification and debugging. Finally, the *calibration* stage incorporates backend-specific information to mitigate device variability [7], [8]. This involves pulse-level tuning and control adjustments on target qubits to achieve high-fidelity execution. Overall, effective OpenQASM programming demands integrated expertise spanning quantum algorithms, software frameworks, and hardware-level calibration.

Large Language Model (LLM)-based agents [12], [13] have demonstrated strong performance in classical code generation across languages such as C, Python, and Verilog [14]–[17], frequently leveraging ReAct-style reasoning [18] and multi-agent coordination [16], [17]. In contrast, their application to quantum computing remains comparatively immature. Current efforts primarily rely on static LLMs—pre-trained, fine-tuned, or prompt-engineered—to address narrowly scoped tasks, including quantum physics problem solving [19], [20], circuit construction [11], [21]–[26], and algorithm explanation [26]. Although such models can generate high-level SDK code (e.g., Qiskit or PennyLane) [27], [28], which can be transpiled into OpenQASM, they do not incorporate hardware calibration into the generation process. Moreover, existing LLM-based quantum agents remain confined to domain-specific applications, including quantum chemistry [29], error correction [30], and experimental automation [31]. Critically, prior approaches fail to support the integrated, multi-stage workflow—encompassing planning, synthesis, and calibration—required for generating executable, hardware-aware OpenQASM programs.

Building such an LLM-based agent system poses three key technical challenges that cannot be addressed by stan-

alone LLMs. First, accurate task planning requires domain-specific knowledge—such as kernel input–output specifications and hardware resource constraints—that is typically absent from general-purpose pretraining corpora. Second, synthesizing valid OpenQASM code necessitates iterative generation, execution, reasoning, and debugging, which exceeds the capabilities of single-pass inference. Third, producing hardware-aware programs requires programmatic access to device-specific calibration data (e.g., qubit frequencies, drift characteristics, and pulse schedules), for which LLMs lack native integration mechanisms. Our observations with Qwen3-235B further substantiate these limitations (Figure 1).

In this paper, we introduce QAgent, the first autonomous multi-agent framework for low-level OpenQASM code generation. QAgent supports the planning–synthesis–calibration workflow by combining explicit knowledge retrieval via RAG [32] with coordinated multi-agent reasoning, testing, and verification. Our contributions are as follows:

- **Schema-aware planning.** We define a structured JSON schema that captures each kernel’s functionality, I/O interface, parameters, and resource requirements. A *Planner* decomposes user intents into executable workflows, while a *Criticizer* iteratively checks schema consistency and agent capability constraints.
- **Example- and tool-aware code synthesis.** Based on the resulting plan, coding tasks are assigned to either the *Guided Few-Shot Coding Agent* (GFCA) for structured kernels or the *Tools-Augmented Coding Agent* (TACA) for parameterized or complex kernels. Both agents incorporate an *Advisor* for retrieval, a *Coder* for synthesis, and a *Reviewer* for simulation-based validation.
- **Hardware-aware calibration.** To account for current device conditions, a *Calibration Agent* parses backend constraint files, including qubit parameters, control ports, frequencies, drift, and pulse schedules, or follows expert directives to produce hardware-aligned OpenQASM code.
- **Integrated evaluation.** Evaluated on twelve representative quantum kernels and their compositions across five LLMs, QAgent improves Pass@1 by 47–70% on single-kernel tasks and achieves over 88% accuracy for large LLMs on multi-kernel benchmarks, whereas baselines reach only 12–30% and generally fail on composed tasks. The source code is available at <https://github.com/fuzhenxiao/QAgent>.

## II. BACKGROUND AND MOTIVATION

### A. OpenQASM Programming

**OpenQASM.** Quantum algorithms are ultimately realized as quantum circuits [34]. Although high-level SDKs such as Qiskit [27] and PennyLane [28] can generate OpenQASM as a hardware-level intermediate representation, the resulting circuits often neglect device-specific constraints, e.g., calibration. Consequently, direct OpenQASM programming remains essential for guiding compiler optimizations [6], enabling pulse-level refinements [7], and supporting hardware-aware calibration and control [8], particularly on NISQ devices [35].

**Workflow.** OpenQASM programming typically follows a structured workflow [1]: (1) *Task decomposition and kernel selection.* The target algorithm is decomposed into subtasks, and appropriate circuit kernels [11] are selected from reference implementations or utility libraries. Standard kernels (e.g., Deutsch–Jozsa) follow canonical templates, whereas complex or parameterized kernels (e.g., quantum adders) often require specialized toolkits, such as register constructors or arithmetic macros. (2) *Circuit drafting.* Gate sequences are specified, registers are initialized, and quantum and classical resources are allocated. (3) *Testing and refinement.* The circuit is iteratively validated and debugged to identify and correct logical and syntactic errors.

**Calibration.** OpenQASM represents quantum programs using abstract operations (e.g., unitary transformations, measurements, and resets), which are ultimately implemented via platform-dependent, time-varying control signals. For example, in superconducting NISQ systems, logical gates are realized via shaped microwave pulses tuned to qubit resonance frequencies. These frequencies can drift by hundreds of kilohertz [36] due to noise and environmental fluctuations [37], leading to reduced gate fidelity. To mitigate this, hardware providers (e.g., IBM) perform periodic recalibration and publish updated *constraint files* containing qubit frequencies, coupling maps, pulse parameters, and device limits [36]. OpenQASM 3 incorporates such hardware specificity through calibration constructs, including `defcal` and `defcalgrammar`, which bind logical operations to calibrated pulse-level implementations. During compilation, gate invocations are automatically resolved using these updated definitions, ensuring hardware-consistent execution.

### B. Current Applications of LLMs in Quantum Tasks

The success of LLMs in classical domains [14]–[17] has motivated their application to quantum computing. Existing efforts can be broadly categorized into *static approaches* and *agentic systems*. Static approaches—encompassing pretraining, fine-tuning, and prompt engineering—aim to enhance model capabilities or better exploit latent knowledge. Prior work has improved LLM performance in circuit optimization and architecture design [21], [23], as well as circuit interpretation [24], [26]. Some static LLMs can generate high-level SDK programs using frameworks such as Qiskit and PennyLane [27], [28], which can be transpiled into OpenQASM; however, they do not incorporate hardware calibration. QCCircuitNet [11] further enhances OpenQASM generation by providing large-scale example corpora. In addition, structured prompting techniques enable stepwise reasoning for quantum physics problem solving [19]. Agentic systems, by contrast, emphasize task decomposition, domain-aware reasoning, and integration with external tools and knowledge sources. For example, EI-Agente [29] analyzes user inputs, selects tools for molecular geometry generation, and returns simulation results, while the framework in [31] employs inspection agents to incorporate external information (e.g., figures) into experimental workflows. These paradigms are complementary rather than

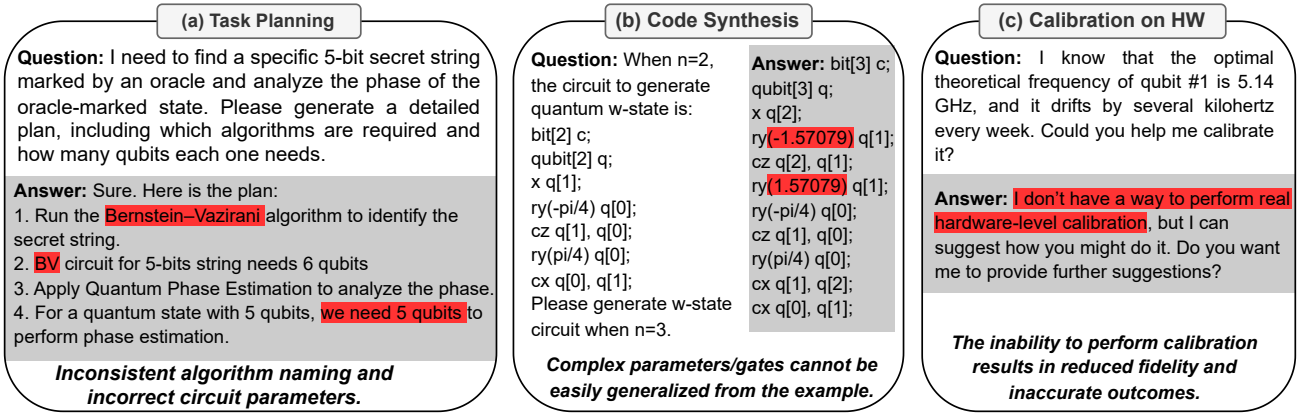


Fig. 1: Evaluating knowledge-intensive OpenQASM code generation on Qwen235B [33] reveals substantial limitations across three stages: (a) task planning, (b) code synthesis, and (c) calibration on hardware.

mutually exclusive; for instance, [30] combines an agentic framework for Qiskit code generation with fine-tuning on public datasets. Despite this progress, existing approaches are largely confined to domain-specific applications or high-level programming tasks. A fully automated OpenQASM programming pipeline—spanning user intent specification, structured task decomposition, hardware-aware code synthesis, and calibration—remains an open challenge.

### C. Design Motivation

We conduct a preliminary evaluation of Qwen3-235B [33], a 235-billion-parameter general-purpose LLM, to assess its capability in OpenQASM code generation. Following recent agentic code-generation paradigms [16], [17], we decompose the workflow into three stages—planning, code synthesis, and validation—mirroring the structure of quantum assembly programming. The results, summarized in Figure 1, indicate that while the model exhibits partial competence, it shows consistent limitations across all stages. In the planning stage, as shown in Figure 1(a), the model can identify relevant functional kernels (e.g., Bernstein-Vazirani and phase estimation) but fails to allocate resources correctly due to insufficient domain knowledge (e.g., a 5-qubit phase estimation circuit requires an additional ancilla qubit). It also inconsistently refers to the same kernel using different naming variants, leading to ambiguity and potential errors. In the code synthesis stage (Figure 1(b)), even when provided with reference examples, the model struggles to generalize to larger problem instances (e.g., failing to derive correct parameter values such as  $\text{ry}(-0.955317)$  and  $\text{ry}(0.955317)$ ). In the validation stage, the model lacks the ability to access or manipulate backend constraint files and therefore cannot perform calibration or adapt programs to hardware-specific conditions. These findings indicate that reliable OpenQASM generation requires an integrated system that combines domain-aware planning with structured kernel representations, supports iterative code synthesis with execution-based feedback, and enables programmatic interaction with hardware calibration data—capabilities not jointly provided by existing LLM-based approaches.

## III. QAGENT

In this section, we present QAgent, an LLM-based multi-agent framework for OpenQASM code generation that spans multiple coordinated stages, including planning, code synthesis, and calibration. We first provide an overview of the system workflow and then describe each stage in detail. QAgent systematically organizes domain knowledge required for OpenQASM programming, including structured programming schemas, kernel-level code examples, language utilities, tool interfaces, and backend constraint specifications. At each stage, relevant information is dynamically retrieved via Retrieval-Augmented Generation (RAG) [32]. The overall reasoning process is orchestrated using the ReAct paradigm [18], coupled with iterative review and refinement mechanisms. This design enables coordinated multi-agent interaction, improving robustness and reliability in end-to-end code generation.

### A. Overall Workflow

As illustrated in Figure 2, QAgent processes natural-language OpenQASM requests through three coordinated stages: task planning, code synthesis, and calibration. The *Planning Agent* interprets the user input, retrieves relevant schemas and examples via the RAG module, and decomposes the request into a graph-structured execution plan that specifies individual subtasks and their dependencies. This plan is then passed to the code synthesis stage, where two complementary agents operate: the *Guided Few-Shot Coding Agent* (GFCA), which handles structured kernels using retrieved exemplars, and the *Tools-Augmented Coding Agent* (TACA), which addresses complex or parameterized kernels through specialized utilities. Each agent follows an iterative advisor-coder-reviewer loop to generate, test, and refine subcircuits. The validated components are subsequently integrated into a unified OpenQASM program. Finally, the *Calibration Agent* incorporates backend constraint files or user-provided calibration directives, producing a hardware-aligned OpenQASM program along with an associated calibration report.

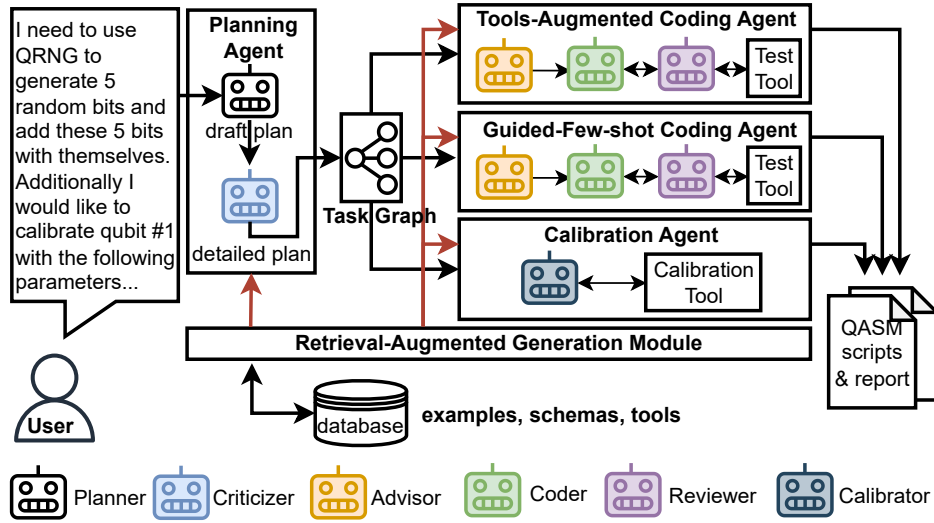


Fig. 2: The workflow of QAgent.

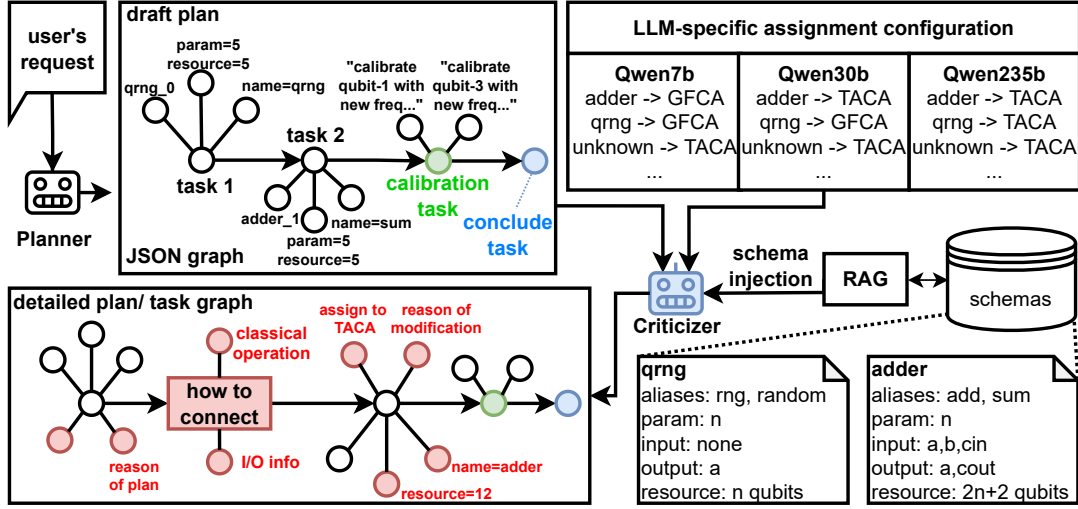


Fig. 3: An illustration of our Planning Agent.

### B. Task Planning

The Planning Agent transforms a high-level user instruction into a structured and executable task graph. Within this module, the LLM operates in two complementary roles: a *planner*, which generates an initial draft, and a *criticizer*, which refines and validates the plan into an executable representation (Figure 3). Given a user request, the planner first decomposes it into a set of subtasks, producing a draft workflow that captures the overall structure but may lack accuracy and consistency. As illustrated in Figure 3, a representative request is partitioned into multiple tasks. For example, one task specifies the generation of a quantum random number generator (QRNG) kernel, including its identifier, parameters, and qubit requirements. Another task, corresponding to an adder kernel, highlights typical deficiencies: the planner may assign an incorrect label (e.g., “sum”) and underestimate required resources (e.g., adding two 5-bit integers requires additional qubits for carry propagation). Additional tasks may include calibration operations, where each node defines the

procedure for calibrating individual qubits, and a terminal aggregation step that consolidates outputs. The draft plan is subsequently refined by the criticizer, which integrates two sources of external knowledge via the RAG module: (1) structured schema definitions that specify standardized kernel semantics, including parameters, I/O interfaces, and resource requirements; and (2) predefined routing policies that determine task allocation across agents (e.g., QRNG  $\rightarrow$  GFCA; adder  $\rightarrow$  TACA). By incorporating schema constraints and routing strategies, the criticizer resolves inconsistencies (e.g., correcting kernel names and resource estimates) and augments the plan with additional semantic annotations. These include explicit agent assignments, rationale for task decomposition, and inter-task dependencies expressed through classical control operations (e.g., measurement and state preparation). The resulting task graph is coherent, validated, and semantically complete, serving as the formal specification for downstream execution.

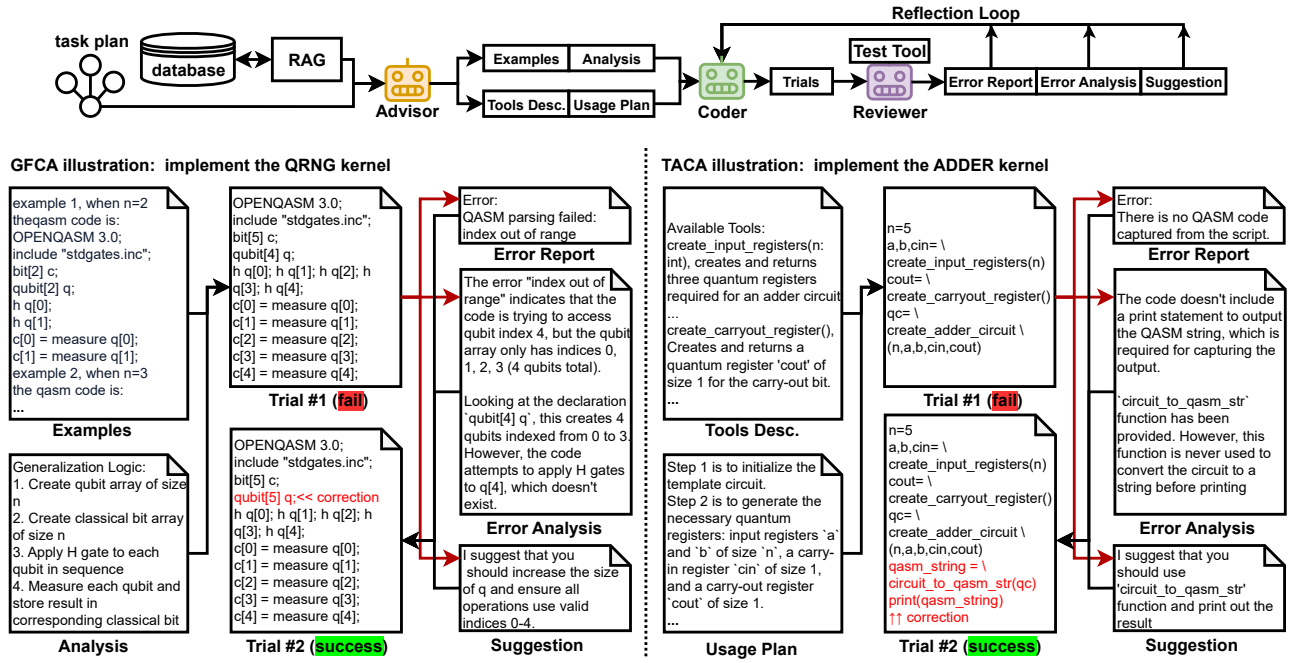


Fig. 4: An illustration of our Coding Agents.

### C. OpenQASM Code Synthesis

Once a task node is dispatched by the Planning Agent, OpenQASM generation proceeds through a unified advisor–coder–reviewer pipeline (Figure 4). Both the Guided Few-Shot Coding Agent (GFCA) and the Tools-Augmented Coding Agent (TACA) follow this iterative, reflection-driven workflow. The *advisor* retrieves task-relevant knowledge via RAG, the *coder* produces an initial implementation, and the *reviewer* executes the program using testing tools and returns structured feedback, including error messages, diagnostics, and revision suggestions. This closed-loop process iterates until a valid OpenQASM program is obtained. Although GFCA and TACA differ in the type of knowledge they leverage (examples versus tool specifications), the shared reflection mechanism enables robust handling of diverse quantum kernels.

1) *Guided Few-shot Coding Agent (GFCA)*: GFCA targets regular, pattern-stable kernels using example-driven reasoning. As shown on the left side of Figure 4, the advisor retrieves multiple annotated examples from the database (e.g., QRNG implementations for various qubit number values of  $n$ ), then analyzes them to infer the general generation logic: create a qubit array, create a classical bit array, apply  $H$  gates, measure sequentially, and store results in the corresponding classical bits. Using this distilled structure, the coder drafts an initial OpenQASM program. The reviewer then executes this program and returns a detailed error report. In the illustrated QRNG example, the first trial fails with a QASM parsing error indicating an “index out of range” condition. The error analysis traces the failure to a mismatch between the number of allocated qubits (four) and the indices accessed (0–4), and recommends increasing the register size and ensuring that all measurement operations fall within bounds. The advisor forwards this suggestion back to the coder, who corrects the

register declaration and regenerates the QASM code. Trial 2 succeeds, concluding the reflection loop. GFCA thus relies on example-derived structural logic, example-specific errors, and example-aware corrections.

2) *Tools-Augmented Coding Agent (TACA)*: TACA handles kernels that depend on modular circuit-construction utilities rather than static structural patterns. As shown on the right side of Figure 4, the advisor retrieves database for tool descriptions and function signatures, such as:

- `create_input_registers()`
- `create_carryout_register()`
- `create_adder_circuit()`
- `circuit_to_qasm_str()`

Then, the advisor synthesizes a tools-usage plan outlining the sequence for assembling the adder circuit. The coder follows this plan to generate a Python script that constructs the full quantum adder. The reviewer then executes this script and analyzes both the produced circuit and its QASM output. In the illustrated case, Trial 1 correctly builds the circuit but fails to output any QASM because the coder omitted the call to `circuit_to_qasm_str()`. The error report identifies that “there is no QASM code captured from the script,” and the error analysis explains that the circuit-to-QASM conversion function must be invoked. The reviewer’s suggestion is passed back to the coder, who adds the missing tool invocation and regenerates the script. Trial 2 succeeds and produces valid QASM. TACA therefore relies on tool catalogs, tool-specific failure modes, and tool-driven corrections.

### D. Calibration

The Calibration Agent bridges abstract program synthesis and hardware-executable OpenQASM. It operates in two



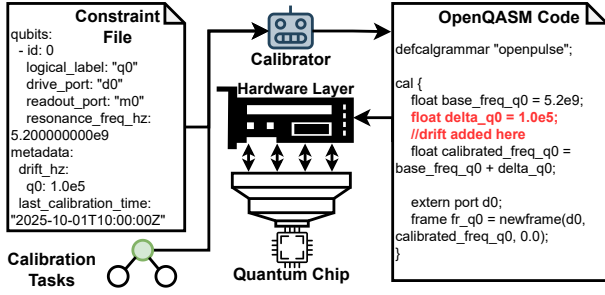


Fig. 5: An illustration of our Calibration Agent

modes depending on available inputs. Expert users may provide explicit calibration directives (e.g., frequency adjustments or control-port mappings), whereas general users rely on backend-supplied constraint files from providers such as IBM. As illustrated in Figure 5, a constraint file specifies per-qubit hardware attributes, including identifiers (e.g., `id: 0`), logical labels (e.g., `q0`), control and readout ports (e.g., `drive_port: d0` and `readout_port: m0`), resonance frequencies, drift parameters (e.g., `drift_hz: 0.1e6`), and calibration timestamps. These parameters define the physical operating conditions under which logical OpenQASM instructions are realized at the pulse level. The Calibration Agent parses constraint files to extract parameters relevant to planned calibration tasks. This includes interpreting qubit frequencies, incorporating drift corrections, and establishing mappings between logical qubits and physical control channels. When user-provided calibration directives are available, the agent integrates them by updating or overriding corresponding hardware parameters. Based on this information, the agent generates hardware-aware OpenQASM (or OpenPulse) calibration code, as depicted in Figure 5, where calibrated frequencies, updated drive settings, and device-level specifications are embedded into the program. The resulting output reflects the current hardware state, including drift effects, port mappings, and calibration metadata, ensuring that the synthesized circuit is aligned with device constraints and suitable for reliable execution on the target quantum processor.

#### IV. EXPERIMENTAL METHODOLOGY

**Benchmarks.** We evaluate QAgent on a representative set of quantum kernels derived from the QCircuitNet dataset [11], with qubit sizes ranging from 3 to 12. The twelve kernels used in this study are listed in the leftmost column of Table I. To illustrate their characteristics, the central columns report detailed gate counts for a subset of kernels instantiated at a circuit scale of 5 qubits. Specifically, we quantify three categories of operations: basic quantum gates (e.g., H, X), composite or custom gates (e.g., inverse QFT), and parameterized gates (e.g., RX). These kernels span a wide spectrum of circuit complexities, ranging from elementary one- and two-qubit operations to advanced constructions involving custom and parameterized gates that require fine-grained control. This diversity facilitates a comprehensive evaluation of both performance and generalization across heterogeneous circuit

TABLE I: Simulated quantum benchmark kernels when  $n = 5$  ( $n$ : qubit count; Q: quantum; Param.: parameterized).

| Benchmarks |                          | Number of Gates ( $n = 5$ ) |        |        | Feature                             |
|------------|--------------------------|-----------------------------|--------|--------|-------------------------------------|
| ID         | Kernel                   | Simple                      | Custom | Param. |                                     |
| 1          | Bernstein–Vazirani (bv)  | 12                          | 0      | 0      | Structured / Oracle                 |
| 2          | Deutsch–Jozsa (dj)       | 12                          | 0      | 0      |                                     |
| 3          | Grover’s Search (gr)     | 27                          | 4      | 0      |                                     |
| 4          | Adder (ad)               | 7                           | 10     | 0      | Composite/Parameterized/Algorithmic |
| 5          | Or Gate (or)             | 58                          | 14     | 48     |                                     |
| 6          | Permutation (pm)         | 4                           | 1      | 0      |                                     |
| 7          | Phase Estimation (pe)    | 22                          | 1      | 0      | Composite                           |
| 8          | Q RNG (qr)               | 5                           | 0      | 0      | Measurement                         |
| 9          | Q Fourier Transform (qf) | 20                          | 0      | 36     | Parameterized                       |
| 10         | W-State (ws)             | 8                           | 0      | 9      | Multipartite                        |
| 11         | GHZ-State (gh)           | 5                           | 0      | 0      |                                     |
| 12         | Cluster-State (cl)       | 9                           | 0      | 0      | Preparation                         |

structures and workloads. To further assess compositional capability, we construct 15 multi-kernel tasks by combining primitive kernels into more complex quantum workflows, with 5 tasks each for 2-, 3-, and 4-kernel compositions. *Notably, for two kernels—GHZ-State and Cluster-State—the corresponding schemas, examples, and tool definitions are automatically generated by Qwen3-235B. This demonstrates the scalability of QAgent in incorporating new kernels without manual intervention.*

**LLMs.** We evaluate QAgent using the following LLMs, and use members of the Qwen family with different parameter scales for the ablation study:

- *Qwen7b*: Qwen2.5-Coder-7B-Instruct [38],
- *Qwen30b*: Qwen3-Coder-30B-A3B-Instruct [33],
- *Qwen235b*: Qwen3-235B-A22B-Instruct-2507 [33],
- *Devstral24b*: Devstral-Small-2505 [39],
- *gpt120b*: gpt-oss-120b [40].

**Schemes.** To evaluate QAgent, we implement and compare the following schemes:

- *Overall performance.* We first evaluate the end-to-end performance of QAgent on both single-kernel and multi-kernel tasks across five LLMs, assessing its ability to complete the full planning–synthesis–calibration workflow as task complexity increases.
- *Component-level ablation.* We then analyze the contribution of individual agents within QAgent. For the Planning Agent, we examine the effect of the proposed schema-aware mechanism on planning accuracy. For the Coding Agents, we compare GFCA and TACA across all twelve kernels against the static baseline [11], which is the closest prior work targeting OpenQASM code generation. For the Calibration Agent, we evaluate circuit fidelity with and without calibration to quantify its impact.
- *Comparison with SDK-oriented LLM methods.* Finally, we compare QAgent with three representative LLM-based approaches for Qiskit and PennyLane code generation: Qiskit-Code-Assistant (QCA) [27], a trained Qiskit-oriented LLM; Pennylang [28], a RAG-based PennyLane generator; and Qiskit-Multi-Agent-System (QMAS) [30], which integrates

TABLE II: Overall performance of QAgent with different base LLMs across multi-kernel settings.

| Pass@k | LLM         | 1-kernel | 2-kernels | 3-kernels | 4-kernels |
|--------|-------------|----------|-----------|-----------|-----------|
| Pass@1 | Qwen7b      | 0.59     | 0.21      | 0.03      | 0.02      |
|        | Qwen30b     | 0.97     | 0.89      | 0.77      | 0.73      |
|        | Qwen235b    | 0.98     | 0.94      | 0.94      | 0.88      |
|        | Devstral24b | 0.87     | 0.65      | 0.34      | 0.26      |
|        | gpt120b     | 0.99     | 1.00      | 0.98      | 1.00      |
| Pass@3 | Qwen7b      | 0.78     | 0.39      | 0.06      | 0.04      |
|        | Qwen30b     | 0.99     | 0.93      | 0.76      | 0.75      |
|        | Qwen235b    | 0.99     | 0.96      | 0.94      | 0.90      |
|        | Devstral24b | 0.97     | 0.81      | 0.50      | 0.44      |
|        | gpt120b     | 1.00     | 1.00      | 0.98      | 1.00      |
| Pass@5 | Qwen7b      | 0.83     | 0.46      | 0.12      | 0.07      |
|        | Qwen30b     | 1.00     | 0.96      | 0.80      | 0.80      |
|        | Qwen235b    | 1.00     | 0.98      | 0.94      | 0.92      |
|        | Devstral24b | 0.99     | 0.82      | 0.52      | 0.46      |
|        | gpt120b     | 1.00     | 1.00      | 0.98      | 1.00      |

TABLE III: Pass@1 results for the Planning Agents.

| Scheme     | Qwen7b       | Qwen30b       | Qwen235b      |
|------------|--------------|---------------|---------------|
| w/o Schema | 0.34         | 0.69          | 0.75          |
| w/ Schema  | 0.35 (2.9%↑) | 0.85 (23.2%↑) | 0.94 (25.3%↑) |

fine-tuning with a multi-agent framework for Qiskit programming. As these methods target high-level SDKs rather than OpenQASM directly, their outputs can only be transpiled into limited OpenQASM representations for simple circuits. Accordingly, we restrict this comparison to relatively simple and structured kernels (bv, dj, and qr) to ensure a fair evaluation based on circuit fidelity.

**Evaluation Metric.** We evaluate model performance using the Pass@k metric [41], where Pass@k denotes the probability that at least one of the  $k$  independently generated candidate solutions is correct:

$$\text{pass}@k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

Where  $n$  is the total number of generated samples for a task and  $c$  is the number of correct ones among them. Smaller values of  $k$  reflect the model’s ability to produce a correct solution within the first few attempts, indicating its per-sample accuracy. We count a generated case as correct only when it is syntactically valid OpenQASM code and its simulated output distribution (1,000 shots) matches the distribution of the golden circuit. Larger values of  $k$  instead capture the model’s upper-bound performance when more sampling is allowed. In our experiments, 10 samples are collected from each LLM for each task, and we report results for Pass@1, Pass@3, and Pass@5. In addition, the execution quality of the final generated code is measured by fidelity, a standard metric for assessing quantum circuit performance.

## V. EXPERIMENTAL RESULTS

### A. Overall Performance

Table II reports the performance of QAgent across different base LLM scales and task complexities, measured by the number of composed quantum kernels.

We first observe that QAgent with **gpt120b** achieves near-perfect accuracy across all kernel counts and Pass@ $k$  settings, with minimal variance, effectively establishing an empirical upper bound. **Devstral24b** also performs strongly on single-kernel tasks, reaching 87% Pass@1 and 99% by Pass@5. These results indicate that large-scale models can fully leverage the multi-agent workflow—including planning, decomposition, retrieval, iterative refinement, and calibration—to maintain high correctness even for complex compositions.

Within the **Qwen** family, three key trends emerge. (1) *Model scale correlates with multi-kernel robustness.* Qwen7B achieves 59% Pass@1 on single-kernel tasks but degrades sharply with increasing complexity, dropping to 2% for four-kernel compositions. Qwen30B shows substantial improvement, achieving 97–100% on single kernels and 73–96% on multi-kernel tasks. Qwen235B is the most stable, maintaining 98–100% on single kernels and strong multi-kernel performance (above 94% for three kernels and above 88% for four kernels). (2) *Larger models exhibit higher sample efficiency.* Increasing  $k$  (Pass@3, Pass@5) yields the largest relative gains for Qwen7B, indicating reliance on sampling to compensate for weaker reasoning. In contrast, Qwen30B and Qwen235B achieve high accuracy with limited sampling. (3) *Planning robustness is the primary limitation for smaller models.* The sharp performance decline of Qwen7B in multi-kernel settings reflects difficulty in decomposing and coordinating multi-stage workflows, whereas larger models demonstrate more reliable multi-step reasoning and stronger alignment between planning and synthesis.

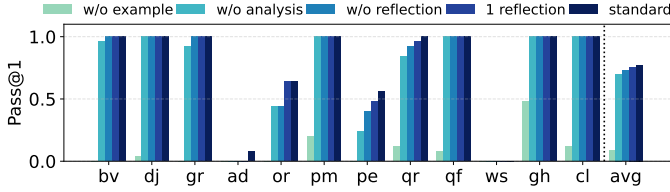
Overall, QAgent consistently improves OpenQASM generation across LLM scales. However, sustained high accuracy on multi-kernel tasks depends on the reasoning capacity of larger models. While smaller models benefit from increased sampling, reliable multi-kernel synthesis requires higher-capability backbones.

### B. Component-Level Ablation Study

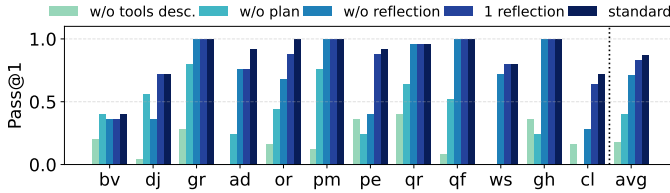
*Planning Agent.* Table III reports Pass@1 planning accuracy with and without the proposed schema-awareness mechanism across model scales. Two observations emerge. (1) Larger models exhibit stronger baseline planning capability: Qwen7B achieves 34%, compared to 69% for Qwen30B and 75% for Qwen235B, reflecting improved intrinsic ability for task decomposition and workflow reasoning. (2) Schema awareness consistently improves accuracy across all models. The gains are most significant for Qwen30B and Qwen235B, with relative improvements of 23.2% and 25.3%, respectively, indicating that structured kernel metadata enhances plan consistency and correctness. In contrast, Qwen7B shows only marginal improvement (2.9%), suggesting limited capacity to utilize structured inputs effectively.

TABLE IV: Pass@1 results comparison for the Coding Agents.

| LLM      | Scheme | bv   | dj   | gr   | ad   | or   | pm   | pe   | qr   | qf   | ws   | gh   | cl   | avg  |
|----------|--------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| Qwen7b   | Static | 0.56 | 0.24 | 0.32 | 0.00 | 0.00 | 0.00 | 0.32 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.12 |
|          | GFCA   | 0.76 | 0.60 | 0.36 | 0.12 | 0.00 | 0.28 | 0.32 | 0.96 | 0.96 | 0.00 | 1.00 | 0.92 | 0.52 |
|          | TACA   | 0.04 | 0.16 | 0.00 | 0.36 | 0.16 | 0.20 | 0.16 | 0.32 | 0.60 | 0.44 | 0.20 | 0.04 | 0.22 |
| Qwen30b  | Static | 0.80 | 0.88 | 0.40 | 0.00 | 0.00 | 0.00 | 0.64 | 0.20 | 0.00 | 0.00 | 0.00 | 0.36 | 0.27 |
|          | GFCA   | 1.00 | 1.00 | 1.00 | 0.08 | 0.64 | 1.00 | 0.56 | 1.00 | 1.00 | 0.00 | 1.00 | 1.00 | 0.77 |
|          | TACA   | 0.40 | 0.72 | 1.00 | 0.92 | 1.00 | 1.00 | 0.92 | 0.96 | 1.00 | 0.80 | 1.00 | 0.72 | 0.87 |
| Qwen235b | Static | 0.88 | 0.92 | 0.44 | 0.04 | 0.08 | 0.04 | 0.72 | 0.00 | 0.04 | 0.00 | 0.00 | 0.40 | 0.30 |
|          | GFCA   | 1.00 | 1.00 | 0.52 | 0.72 | 0.52 | 1.00 | 0.88 | 0.96 | 1.00 | 0.12 | 1.00 | 1.00 | 0.81 |
|          | TACA   | 1.00 | 0.92 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 0.96 | 0.80 | 0.96 | 0.92 | 0.96 |



(a) Pass@1 results across different GFCA configurations.



(b) Pass@1 results across different TACA configurations.

Fig. 6: Ablation study of the two Coding Agents.

*Coding Agents.* Table IV summarizes Pass@1 performance for three coding strategies: Static prompting [11], GFCA-only, and TACA-only. Two conclusions follow. First, static prompting is insufficient for reliable OpenQASM generation, particularly for multi-step or tool-dependent kernels. Second, both GFCA and TACA substantially improve correctness: GFCA performs best on structured kernels, while TACA achieves superior results on complex or tool-intensive kernels, especially for larger models. Strong performance on *gh* and *cl* further indicates that QAgent generalizes to kernels whose schemas and tools are LLM-generated.

*GFCA Ablation.* Figure 6a evaluates key components of GFCA. (1) Removing example retrieval reduces average accuracy from 77.3% to 8.7%, demonstrating that representative examples are essential for capturing recurring structural patterns (e.g., *bv*, *dj*, *qr*). (2) Removing example analysis decreases accuracy to 70.0%, indicating that abstracting shared structural logic is necessary for generalization. (3) Removing reflection lowers performance to 73.0%, while limiting to a single iteration yields only 75.7%, confirming that iterative refinement is important for correcting subtle logical and syntactic errors. These trends are consistent across kernels and align with overall performance results.

*TACA Ablation.* Figure 6b highlights the importance of TACA components. (1) Removing tool descriptions reduces

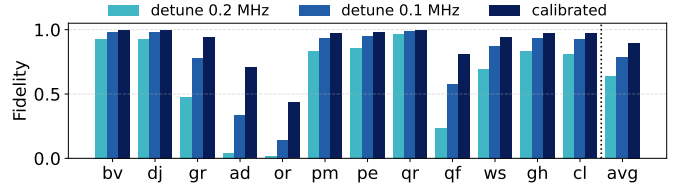


Fig. 7: Fidelity performance of the Calibration Agent.

accuracy to 18.0%, showing that explicit API-level knowledge is critical for tool-dependent kernels (e.g., *ad*, *or*, *pm*, *pe*). (2) Removing the usage plan results in 40.3% accuracy, underscoring the need for structured orchestration of tool calls. (3) Removing reflection decreases accuracy from 87.0% to 71.0%, and limiting to a single iteration (83.3%) still underperforms the full configuration, indicating that iterative validation is essential for resolving execution and I/O issues. These findings are consistent with Table IV, where TACA outperforms GFCA on tool-intensive kernels.

*Summary on Coding Agents.* Overall, the combined evidence from the performance table and the ablation studies validates the complementary design of the two agents. GFCA is driven by example abstraction and structural pattern inference, enabling strong performance on regular kernels. TACA depends on tool-aware retrieval, procedural planning, and iterative correction, making it well-suited for complex or parameterized kernels. Reflection consistently enhances robustness in both agents, confirming its role as a key mechanism for reliable OpenQASM generation within QAgent.

*Calibration Agent.* We simulate realistic hardware detuning scenarios on superconducting backends in which each qubit's resonance frequency is shifted by 0.1–0.2 MHz from its nominal value. Such frequency drift is common in NISQ-era devices and, if uncorrected, can significantly reduce gate fidelity and overall circuit correctness [36]. Figure 7 reports the resulting circuit fidelities across the 12 benchmark kernels. Under a 0.2 MHz detuning, fidelity degrades substantially, dropping to an average of 63%. Kernels with greater two-qubit gate depth (such as Grover and Adder) are particularly affected, falling to roughly 48% and 4%, respectively. Even with a milder 0.1 MHz drift, the average fidelity remains only 78%, showing that small but persistent hardware deviations can meaningfully impair execution reliability. Once the Calibration Agent parses



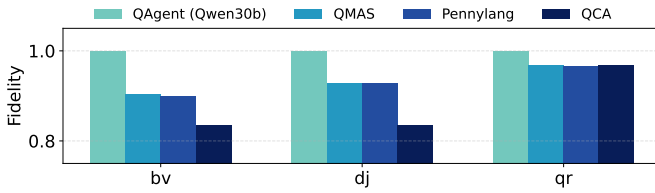


Fig. 8: Average fidelity comparison with SDK-oriented LLM methods under 0.2 MHz frequency detuning (10 attempts, 1000 shots each).

the hardware constraint file and inserts calibrated frequency parameters into the generated OpenQASM/OpenPulse program, execution fidelity rebounds across all kernels. The calibrated configuration achieves an average fidelity of 89%, with high-depth and parameter-sensitive circuits exhibiting improvements exceeding 30 percentage points. These results demonstrate that automated, hardware-aware calibration is essential for mitigating frequency drift and preserving reliable quantum program execution.

### C. Comparison against SDK-oriented LLMs

Figure 8 reports the average circuit fidelity (10 runs with 1,000 shots each) of QAgent and three SDK-oriented baselines on three structured kernels under a uniform 0.2 MHz frequency detuning. QAgent (with Qwen30B) achieves near-unity fidelity across all kernels (above 0.99), whereas the baselines range from 0.83 to 0.97. Among the baselines, QCA exhibits the lowest fidelity, reflecting occasional inaccuracies in interpreting user intent and generating correct circuits, consistent with its role as an assistive tool for Qiskit APIs rather than an end-to-end synthesis system. Pennylang and QMAS achieve higher fidelity by reliably generating correct circuits for these relatively simple kernels. The evaluated kernels are intentionally simple to reduce circuit-generation difficulty across systems, so the remaining fidelity differences mainly reflect whether calibration is applied. Specifically, SDK-oriented methods generate circuits that are transpiled and executed without hardware-specific frequency correction, leaving them vulnerable to device-level detuning. In contrast, QAgent incorporates a calibration stage that parses backend constraint files and produces pulse-level OpenQASM code with drift compensation. This capability effectively mitigates detuning-induced errors. These results highlight a key advantage of QAgent: by integrating hardware-aware calibration into the code generation pipeline, it preserves execution fidelity under realistic hardware conditions, extending beyond circuit-level correctness to reliable physical execution.

## VI. CONCLUSION

In this paper, we present QAgent, the first autonomous multi-agent framework for generating hardware-aligned OpenQASM programs. By integrating schema-aware planning, kernel-guided synthesis, and constraint-file-driven calibration, QAgent enables reliable, end-to-end quantum program

generation that adapts to device variability. Our evaluation across twelve representative kernels demonstrates substantial improvements over existing LLM-based methods, particularly on multi-kernel compositions. As NISQ systems continue to advance, we expect QAgent to serve as a foundation for scalable, hardware-aware quantum software automation. In future work, we plan to extend the framework to richer pulse-level control, dynamic circuit features, and broader quantum hardware platforms.

## ACKNOWLEDGMENT

This work was supported in part by NSF OAC-2417589 and NSF CNS-2143120. We thank the IBM Quantum Researcher & Educators Program for their support of Quantum Credits. Any opinions, findings and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of grant agencies or their contractors.

## REFERENCES

- [1] A. Cross, A. Javadi-Abhari, T. Alexander, N. De Beaudrap, L. S. Bishop, S. Heidel, C. A. Ryan, P. Sivarajah, J. Smolin, J. M. Gambetta, *et al.*, “Openqasm 3: A broader and deeper quantum assembly language,” *ACM Transactions on Quantum Computing*, vol. 3, no. 3, pp. 1–50, 2022.
- [2] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, “Quantum computing with Qiskit,” 2024.
- [3] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, S. Ahmed, V. Ajith, M. S. Alam, G. Alonso-Linaje, B. AkashNarayanan, A. Asadi, *et al.*, “PennyLane: Automatic differentiation of hybrid quantum-classical computations,” *arXiv preprint arXiv:1811.04968*, 2018.
- [4] D. Castelvecchi, “IBM releases first-ever 1,000-qubit quantum chip,” *Nature*, vol. 624, no. 7991, pp. 238–238, 2023.
- [5] S. A. Moses, C. H. Baldwin, M. S. Allman, R. Ancona, L. Ascarrunz, J. Barnes, J. Bartolotta, B. Bjork, P. Blanchard, M. Bohn, *et al.*, “A race-track trapped-ion quantum processor,” *Physical Review X*, vol. 13, no. 4, p. 041052, 2023.
- [6] A. Litteken, Y.-C. Fan, D. Singh, M. Martonosi, and F. T. Chong, “An updated llvm-based quantum research compiler with further openqasm support,” *Quantum Science and Technology*, vol. 5, no. 3, p. 034013, 2020.
- [7] D. C. McKay, T. Alexander, L. Bello, M. J. Biercuk, L. Bishop, J. Chen, J. M. Chow, A. D. Córcoles, D. Egger, S. Filipp, *et al.*, “Qiskit backend specifications for openqasm and openpulse experiments,” *arXiv preprint arXiv:1809.03452*, 2018.
- [8] N. Khammassi, I. Ashraf, J. Someren, R. Nane, A. M. Krol, M. A. Rol, L. Lao, K. Bertels, and C. G. Almudever, “Openql: A portable quantum programming framework for quantum accelerators,” *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, vol. 18, no. 1, pp. 1–24, 2021.
- [9] Amazon Braket Team, “Amazon Braket Examples,” <https://github.com/amazon-braket/amazon-braket-examples>, 2024.
- [10] N. C. L. Ramalho, H. Amario de Souza, and M. Lordello Chaim, “Testing and debugging quantum programs: The road to 2030,” *ACM Transaction on Software Engineering Methodology*, Jan. 2025.
- [11] R. Yang, Y. Gu, Z. Wang, Y. Liang, and T. Li, “Qcircuitnet: A large-scale hierarchical dataset for quantum algorithm design,” *arXiv preprint arXiv:2410.07961*, 2024.
- [12] H. Tang, D. Key, and K. Ellis, “Worldcoder, a model-based llm agent: Building world models by writing code and interacting with the environment,” in *Advances in Neural Information Processing Systems*, vol. 37, pp. 70148–70212, 2024.
- [13] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, “Executable code actions elicit better LLM agents,” in *Forty-first International Conference on Machine Learning*, 2024.
- [14] microsoft, “Building applications with github copilot agent mode,” <https://learn.microsoft.com/en-us/training/modules/github-copilot-agent-mode/>, 2025.

- [15] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese, and Y. Zhou, "Codegen2: Lessons for training llms on programming and natural languages," *arXiv preprint arXiv:2305.02309*, 2023.
- [16] P. Zehua, H. Zhen, M. Yuan, Y. Huang, and B. Yu, "Betternv: Controlled verilog generation with discriminative guidance," in *Forty-first International Conference on Machine Learning*, 2024.
- [17] C.-T. Ho, H. Ren, and B. Khailany, "Verilogcoder: Autonomous verilog coding agents with graph-based planning and abstract syntax tree (ast)-based waveform tracing tool," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, pp. 300–307, 2025.
- [18] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *The eleventh international conference on learning representations*, 2022.
- [19] H. Pan, N. Mudur, W. Taranto, M. Tikhonovskaya, S. Venugopalan, Y. Bahri, M. P. Brenner, and E.-A. Kim, "Quantum many-body physics calculations with large language models," *Communications Physics*, vol. 8, no. 1, p. 49, 2025.
- [20] S. Zhou, R. Chen, Z. An, C. Zhang, and S.-Y. Hou, "Application of large language models to quantum state simulation," *Science China Physics, Mechanics & Astronomy*, vol. 68, no. 4, p. 240313, 2025.
- [21] L. Jern, V. Uotila, C. Yu, and B. Zhao, "Fine-tuning large language models on quantum optimization problems for circuit generation," *arXiv preprint arXiv:2504.11109*, 2025.
- [22] K. Nakaji, L. B. Kristensen, J. A. Campos-Gonzalez-Angulo, M. G. Vakili, H. Huang, M. Bagherimehrab, C. Gorgulla, F. Wong, A. McCaskey, J.-S. Kim, *et al.*, "The generative quantum eigensolver (gqe) and its application for ground state search," *arXiv preprint arXiv:2401.09253*, 2024.
- [23] Z. Liang, J. Cheng, R. Yang, H. Ren, Z. Song, D. Wu, X. Qian, T. Li, and Y. Shi, "Unleashing the potential of llms for quantum computing: A study in quantum architecture design," *arXiv preprint arXiv:2307.08191*, 2023.
- [24] S. Kashani, "Quantumllminstruct: A 500k llm instruction-tuning dataset with problem-solution pairs for quantum computing," *arXiv preprint arXiv:2412.20956*, 2024.
- [25] P. Sinha, S. K. Jha, and S. Raj, "Circuit partitioning using large language models for quantum compilation and simulations," *arXiv preprint arXiv:2505.07711*, 2025.
- [26] G. d'Aloisio, S. Fortz, C. Hanna, D. Fortunato, A. Bensoussan, E. n. Mendiluze Usandizaga, and F. Sarro, "Exploring llm-driven explanations for quantum algorithms," in *ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, (New York, NY, USA), pp. 475–481, Association for Computing Machinery, 2024.
- [27] N. Dupuis, L. Buratti, S. Vishwakarma, A. V. Forrat, D. Kremer, I. Faro, R. Puri, and J. Cruz-Benito, "Qiskit code assistant: Training llms for generating quantum computing code," in *IEEE LLM Aided Design Workshop*, pp. 1–4, 2024.
- [28] A. Basit, N. Innan, H. Asif, M. Shao, M. Kashif, A. Marchisio, and M. Shafique, "Pennylang: Pioneering llm-based quantum code generation with a novel pennylane-centric dataset," *arXiv preprint arXiv:2503.02497*, 2025.
- [29] Y. Zou, A. H. Cheng, A. Aldossary, J. Bai, S. X. Leong, J. A. Campos-Gonzalez-Angulo, C. Choi, C. T. Ser, G. Tom, A. Wang, *et al.*, "El agente: An autonomous agent for quantum chemistry," *arXiv preprint arXiv:2505.02484*, 2025.
- [30] C. Campbell, H. M. Chen, W. Luk, and H. Fan, "Enhancing llm-based quantum code generation with multi-agent optimization and quantum error correction," in *ACM/IEEE Design Automation Conference*, 2025.
- [31] S. Cao, Z. Zhang, M. Alghadeer, S. D. Fasciati, M. Piscitelli, M. Bakr, P. Leek, and A. Aspuru-Guzik, "Agents for self-driving laboratories applied to quantum computing," 2025.
- [32] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in neural information processing systems*, vol. 33, pp. 9459–9474, 2020.
- [33] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu, "Qwen3 technical report," 2025.
- [34] M. A. Nielsen and I. L. Chuang, *Quantum computation and quantum information*. Cambridge university press, 2010.
- [35] F. T. Chong, D. Franklin, and M. Martonosi, "Programming languages and compiler design for realistic quantum hardware," *Nature*, vol. 549, no. 7671, pp. 180–187, 2017.
- [36] C. Xu and J. Szefer, "Long-term analysis of the dependability of cloud-based nisq quantum computers," in *Proceedings of the 18th International Conference on Availability, Reliability and Security*, pp. 1–6, 2023.
- [37] A. A. Clerk, M. H. Devoret, S. M. Girvin, F. Marquardt, and R. J. Schoelkopf, "Introduction to Quantum Noise, Measurement, and Amplification," *Reviews of Modern Physics*, vol. 82, Apr 2010.
- [38] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, Y. Fan, Y. Zhang, A. Yang, R. Men, F. Huang, B. Zheng, Y. Miao, S. Quan, Y. Feng, X. Ren, X. Ren, J. Zhou, and J. Lin, "Qwen2.5-coder technical report," 2024.
- [39] A. Rastogi *et al.*, "Devstral: Fine-tuning language models for coding agent applications," 2025.
- [40] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," 2025.
- [41] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.